Worksheet: Uninformed & Heuristic Search

Foundations of Artificial Intelligence – SS 2026
University of Bremen, Institute for Artificial Intelligence

> **Instructions:** This worksheet accompanies the Python Jupyter Notebook (`search_exercises.ipynb`). Conceptual questions (marked ✏️) can be answered here or in a notebook Markdown cell. Code tasks (marked 🐍) belong in the notebook. Estimated time: **~60 minutes**.

Part 1 – Specializations of Search Algorithms ✏️ *(~10 min)*

Search algorithms can often be seen as special cases of more general ones. Answer the following questions in your own words:

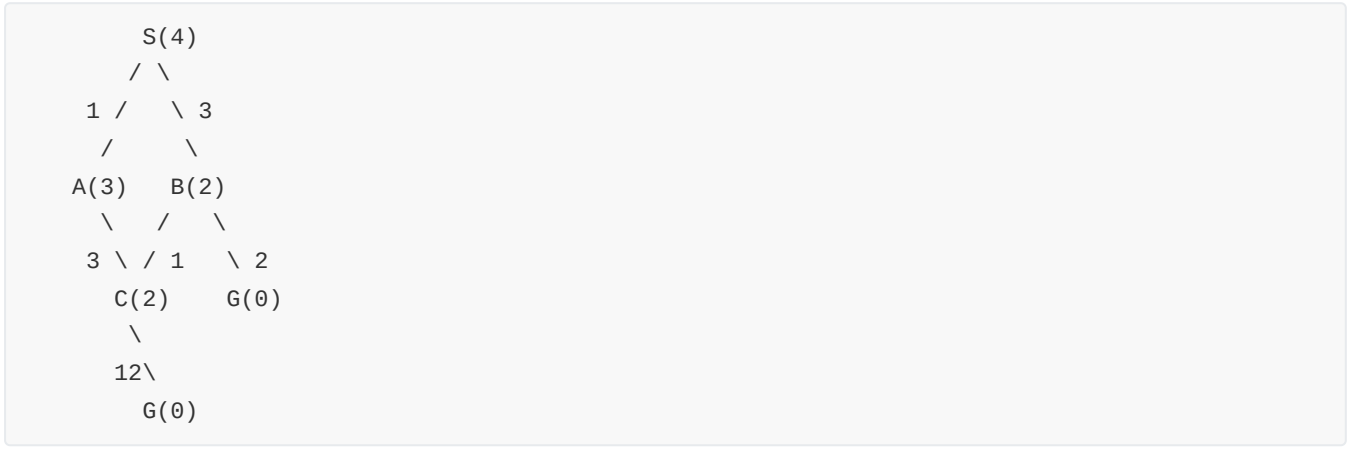
1.1 How can **Breadth-First Search (BFS)** be seen as a special case of **Uniform Cost Search (UCS)**?
(Hint: Think about what step costs would make UCS behave exactly like BFS.)

1.2 How are BFS, Depth-First Search (DFS), and UCS all special cases of **Best-First Search**?
(Hint: What evaluation function `f(n)` does each one use?)

1.3 How is **UCS** a special case of **A* Search**?
(Hint: What heuristic `h(n)` would make A reduce to UCS?)*

Part 2 – Trace UCS and A* by Hand ✏️ *(~15 min)*

Consider the graph below. Numbers in **brackets** are heuristic estimates `h(n)`; numbers on edges are step costs.



Graph edges and costs:

- S → A: cost 1, S → B: cost 3
- A → C: cost 3, B → C: cost 1, B → G: cost 2
- C → G: cost 12

Heuristic values: h(S)=4, h(A)=3, h(B)=2, h(C)=2, h(G)=0

(When multiple nodes have equal priority, expand alphabetically.)

2.1 Apply **Uniform Cost Search (graph-search variant)** to find the path from S to G.
Fill in the table as you go:

Step	Node Expanded	Frontier (node / g-cost)
1		
2		
...		

- Which path was found? What is its total cost?

2.2 Apply **A* Search (graph-search variant)** using `f(n) = g(n) + h(n)`.
Fill in the same kind of table.

- Which path was found? How does it compare to UCS?

2.3 Apply **Greedy Best-First Search** using only `f(n) = h(n)`.

- Which path does it find? Is it optimal? Why or why not?

2.4 Now change `h(C) = 12`. Run A* again.

- Is the heuristic still **admissible**? Does A* still find the optimal path? Explain.

Part 3 – Python: Implementing BFS and DFS 🐍 *(~15 min)*

Open the notebook `search_exercises.ipynb`. In **Exercise 3** you will find a graph represented as a Python dictionary and two incomplete functions.

Your tasks:

3.1 Complete the `bfs(graph, start, goal)` function.

3.2 Complete the `dfs(graph, start, goal)` function.

3.3 Run both on the provided graph and print the path found and how many nodes were visited. Are the paths the same? Why might they differ?

Part 4 – Python: Implementing UCS 🐍 *(~10 min)*

In **Exercise 4** of the notebook you will find a weighted graph and an incomplete `ucs(graph, start, goal)` function that uses a priority queue.

Your tasks:

4.1 Complete `ucs(graph, start, goal)` using Python's `heapq` module (already imported for you).

4.2 Test your implementation on the graph from Part 2. Does it return the same path you traced by hand?

Part 5 – Python: Implementing A* 🐍 *(~10 min)*

In **Exercise 5** of the notebook, the heuristic dictionary and an incomplete `astar(graph, heuristic, start, goal)` function are provided.

Your tasks:

5.1 Complete `astar` using `f(n) = g(n) + h(n)`.

5.2 Run it on the Part 2 graph and compare results with your UCS implementation.

5.3 (Bonus): Change the heuristic so that `h(C) = 12` and run A* again. Does it still find the optimal path?

Part 6 – Reflection ✏️ *(~5 min)*

Answer in 2–3 sentences each:

6.1 What is the key difference between an **admissible** and a **consistent (monotone)** heuristic?

6.2 In which situations would you prefer A* over UCS?

6.3 Why does the **graph-search variant** of A* require a consistent heuristic to guarantee optimality, while the **tree-search variant** only requires admissibility?

Good luck! 🎓